

Towards Taming Data Comparisons in a Description Logic Setting

Valentin Cassano^{1,2}[0000–1111–2222–3333] and Raul Fervari^{1,3}[1111–2222–3333–4444]

¹ Consejo Nacional de Investigaciones Científicas y Técnicas (CONICET), Argentina

² Universidad Nacional de Río Cuarto, Argentina

³ Universidad Nacional de Córdoba, Argentina

Abstract. We study a fragment of the query language XPath featuring data (in)equality operators, and its connection with Description Logics (DL) with Concrete Domains. XPath has already been investigated from a Modal Logic (ML) perspective. Given the connection between ML and DL, it is only natural to look at XPath from the perspective of DL. We resort to DL with concrete domains to naturally capture data comparisons. In doing so, we establish an interesting connection between the two universes, from which we can obtain transference results (e.g., complexity bounds for reasoning tasks). Furthermore, we refine complexity results and show that concept satisfiability, ABox consistency, and concept satisfiability w.r.t. a TBox are all NP-complete problems for the fragment of XPath under consideration. In this way, we get a DL fragment with data comparisons featuring good computational properties.

Keywords: First keyword · Second keyword · Another keyword.

1 Introduction

Data graphs are, essentially, relational structures where nodes contain data values. This type of graph is the fundamental model underpinning two areas of great interest in knowledge representation and web data exchange: graph databases (see, e.g., [?,?]), and XML documents [?] (over the data graphs with a tree structure). For this reason, there is a realm of languages to formally describe and query data graphs. Two examples of this claim are Regular Queries [?], a well-behaved fragment of the Datalog language, and XPath [?], the most prominent approach for querying XML documents. The latter has received much attention lately, as it allows both the description of the topology of a data graph, and manipulation of its data values [?].

It has been argued that XPath can be seen, from a formal perspective, as a modal language [?,?]. This makes it possible to define and use logical tools to reason about data graphs, see e.g. [?,?,?,?,?]. The navigational part of XPath is much like standard Modal Logics, such as Propositional Dynamic Logic (PDL) [?], since it describes the interconnection between nodes in the data graph

via so-called *path expressions*. For instance, the path expression $((ab) \cup c)$ describes a connection established via the composition of an edge labelled by a with one labelled by b , or via an edge labelled by c . Path expressions are used to build *node expressions*. Particularly interesting are those describing data comparisons. These node expressions have the form $\langle \alpha =_e \beta \rangle$, and state that starting from a given node of the graph, there is a node n reached via an α -path, and a node m reached via a β -path (with α and β path expressions), such that the data value contained in n equals the data value of m , for a data comparison type e . The ML perspective has provided results like complexity analysis, expressivity bounds, reasoning algorithms, and complete axiomatizations (see [?, ?, ?, ?, ?] for further details).

Considerably less attention has been put in using XPath as a Description Logic (DL) [?, ?]. It is well-known that ML and DL are strongly related. Thus it is only natural to consider XPath from a DL perspective. In [?, ?], first steps towards investigating XPath as an ontology language were taken. In particular they use DL-Lite reasoners in order to solve XPath queries. However, the fragment of XPath being considered contains only navigational expressivity, and leaves out of the picture data comparisons (a distinctive feature of XPath).

In this article we take one step further, and consider a fragment of XPath featuring data equality and inequality comparisons, in a DL setting. Concretely, we will consider the fragment of XPath in which path expressions are the composition of simple steps (setting aside tests and non-deterministic choice). For node expressions we allow the full power of XPath with data equality and inequality. Moreover, path expressions will be interpreted as partial functions. We call this fragment DXPath_1^n , borrowing notation from the modal logic $K_n\text{Alt}_1$ [?], featuring n modal operators where each respective accessibility relation is a partial function. As we will discuss, these restrictions have an important computational effect, greatly reducing the complexity of different DXPath_1^n reasoning tasks.

In spite of the imposed restrictions, DXPath_1^n is expressive enough to model many interesting situations. Consider a simplified graph database with information about bank accounts (??). This example is inspired by one presented in [?]. Each node in the example contains information on persons, accounts and banks. Each person has associated a data value *ld*, and can own one or more accounts. In [?] this fact is represented by a binary relation connecting a person with her/his respective accounts. Here, the different accounts owned by a person are represented as partial functions labelled as *primary account*, *secondary account*, etc. (of course, this assumes there is a fix bound n to the maximum number of accounts a person can have). Thus, a person's primary account is given by the functional relation *prim.acc*, while the secondary account is given by the functional relation *sec.acc*. Bank accounts have also an *ld* data value. For instance, the person whose *ld* is 6365988, has a primary account with *ld*: 13209 and a secondary account with *ld*: 004569. Account number 712340 is a joint account, this may be the case, e.g., for married people. Bank accounts belong to a unique bank, identified by a data value *Name*. In our example there are two possible bank names, *IB* and *NB* (standing for International and

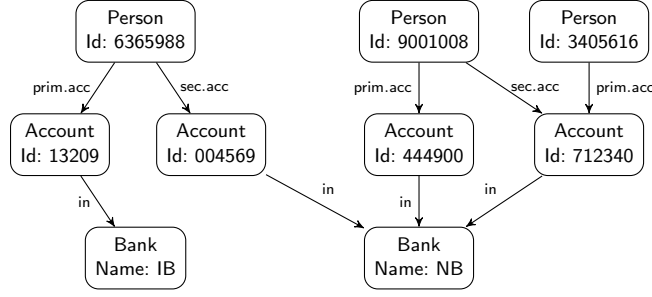


Fig. 1: Example of a graph-structured database.

National Bank, respectively). For a given person, we can express for instance whether the primary and secondary accounts, if exist, have different Id data value with the expression $\langle \text{prim.account} \neq_{\text{Id}} \text{sec.account} \rangle$. Moreover, the expression $\langle \text{prim.account in} =_{\text{Name}} \text{sec.account in} \rangle$ states that a person has in fact a primary and a secondary account, both in the same bank. In the example, this is the case for the person with Id: 9001008.

In addition to showing that DXPath_1^n is expressive enough to capture interesting scenarios (particularly, with functional relations), the example above also serves to illustrate how this logic can be used in a DL setting. For instance, we can include definitions in a TBox and redefine long queries as short ones. To illustrate this point, a definition $\text{elig} \doteq \langle \text{prim.account in} =_{\text{Name}} \text{sec.account in} \rangle$ could be used to define those persons that are eligible for certain bank benefits, whenever they have a primary and a secondary account in the same bank. With this definition, the new concept *elig* can be used as a succinct and more natural way to query the data graph, about such an eligibility criteria.

One way to handle data structures like the ones just described in DL is via so-called concrete domains [?]. However, it has been shown that the inclusion of concrete domains easily increment the computational complexity of DL reasoning tasks (see, e.g., [?]). Thus, our first result is to show that DXPath_1^n can be seen as a DL with a very simple concrete domain. As a result, we can transfer upper bounds for DL reasoning tasks in DXPath_1^n as a by product. It turns out that these bounds are not tight. Our main technical result is to show that the problems of concept satisfiability, ABox consistency and concept satisfiability w.r.t. a TBox are all NP-complete. Such relatively low complexity bounds are surprising, as these reasoning tasks has been prove to be at least PSpace-hard, and in many cases ExpTime-hard or above for many DL with simple concrete domains [?].

Outline. We start in ?? by introducing the logic DXPath_1^n , i.e., a fragment of XPath interpreted over models with functional accessibility relations. In ?? we present the main ingredients of the DL with concrete domains $\mathcal{ALC}(\mathcal{D})$, and

recall some complexity results for common reasoning tasks. Then, in ?? we define an equivalence preserving translation from DXPath_1^n to $\mathcal{ALC}(\mathcal{D})$ (with $\mathcal{D}$ a very simple concrete domain). This translation enables us to obtain some transference results, in particular, regarding the complexity of reasoning tasks. These results are refined in ?. Therein, we establish tight complexity bounds for three typical problems in DL over DXPath_1^n . Precisely, we prove that concept satisfiability, ABox consistency and concept satisfiability w.r.t. a TBox are all NP-complete. Our method is based on a selection function to obtain a *polynomial size model property*. We end this work with some comments and future lines of research in ?.

2 Functional XPath with Data: DXPath_1^n

In this section we introduce a fragment of the language considered in, e.g., [?,?,?,?], which we call DXPath_1^n . The fragment is inspired by the modal logic $K_n\text{Alt}_1$ which has n modal operators interpreted as partial functions [?]. In brief, DXPath_1^n is a two-sorted language which consists of *path* expressions and *node* expressions. Regarding path expressions, we restrict our attention to *single steps* and *composition*, excluding *test* and *choice*. Moreover, paths expressions will be interpreted as partial functions. We do not impose restrictions on node expressions. As mentioned, these design decisions are aimed at establishing connections with some DL with concrete domains.

In what follows, we assume that $\text{Prop} = \{p_1, \dots, p_n, p, q, r, \dots\}$ is a countable set of *propositional symbols*, $\text{Mod} = \{a_1, \dots, a_n\}$ is a finite set of *modal labels*, and $\text{Cmp} = \{e_1, \dots, e_m\}$ is a finite set of *comparison labels*.

Definition 1 (Language). *The set PExp of path expressions (written $\alpha, \beta, \dots$) and NExp of node expressions (written $\varphi, \psi, \dots$) of DXPath_1^n are defined by the grammar:*

$$\begin{aligned} \alpha, \beta &::= a \mid \alpha\beta \\ \varphi, \psi &::= p \mid \neg\varphi \mid \varphi \wedge \psi \mid \langle \alpha =_e \beta \rangle \mid \langle \alpha \neq_e \beta \rangle, \end{aligned}$$

where $p \in \text{Prop}$, $a \in \text{Mod}$, $e \in \text{Cmp}$. Other Boolean connectives are defined as usual. We also define $[\alpha =_e \beta]$ as $\neg\langle \alpha \neq_e \beta \rangle$ and $[\alpha \neq_e \beta]$ as $\neg\langle \alpha =_e \beta \rangle$.

Notice that path expressions are simply elements from Mod^+ . This inductive definition in terms of composition will be handy in some proofs. Now we introduce the structures in which we evaluate DXPath_1^n expressions.

Definition 2 (Data Model). *A data model is a structure defined as $\mathfrak{M} = \langle N, D, \{R_a\}_{a \in \text{Mod}}, V, \{\text{data}_e\}_{e \in \text{Cmp}} \rangle$, where:*

1. N is a non-empty set of nodes;
2. D is a non-empty set of data values;
3. for each $a \in \text{Mod}$, R_a is a functional relation on N ;
4. $V : \text{Prop} \rightarrow 2^N$ is the valuation function;

5. $\text{data}_e : N \rightarrow D$ is a function which assigns a data value to each node of the model with respect to $e \in \text{Cmp}$.

If $(n, m) \in R_a$, sometimes we will use $R_a(n)$ to denote m (recall that R_a is functional) and we say that $R_a(n)$ is defined. If such an m does not exist we say $R_a(n)$ is undefined.

Here, data models are a particular case of the data graphs from, e.g., [?], in which accessibility relations are partial functions.

In general, the set D of data values can be arbitrary, with the only condition imposed that it should be non-empty. In what follows, and to properly establish the connection between XPath and DL with concrete domains, we will assume that a given D is fixed, containing all the data values that we need to consider, and the class of models is defined w.r.t. that fixed set of data values.

Definition 3 (Semantics). Let $\mathfrak{M}$ be a data model such that $\mathfrak{M} = \langle N, D, \{R_a\}_{a \in \text{Mod}}, V, \{\text{data}_e\}_{e \in \text{Cmp}} \rangle$. We define the semantics of path and node expressions as:

$$\begin{aligned}
\llbracket a \rrbracket^{\mathfrak{M}} &= R_a \\
\llbracket \alpha\beta \rrbracket^{\mathfrak{M}} &= \{(m, n) \mid \text{there is } l \text{ s.t.} \\
&\quad (m, l) \in \llbracket \alpha \rrbracket^{\mathfrak{M}} \text{ and } (l, n) \in \llbracket \beta \rrbracket^{\mathfrak{M}}\} \\
\llbracket p \rrbracket^{\mathfrak{M}} &= V(p) \\
\llbracket \neg\varphi \rrbracket^{\mathfrak{M}} &= N \setminus \llbracket \varphi \rrbracket^{\mathfrak{M}} \\
\llbracket \varphi \wedge \psi \rrbracket^{\mathfrak{M}} &= \llbracket \varphi \rrbracket^{\mathfrak{M}} \cap \llbracket \psi \rrbracket^{\mathfrak{M}} \\
\llbracket \langle \alpha =_e \beta \rangle \rrbracket^{\mathfrak{M}} &= \{m \mid \text{there are } n, l \text{ s.t.} \\
&\quad (m, n) \in \llbracket \alpha \rrbracket^{\mathfrak{M}} \text{ and } (m, l) \in \llbracket \beta \rrbracket^{\mathfrak{M}} \text{ and} \\
&\quad \text{data}_e(n) = \text{data}_e(l)\} \\
\llbracket \langle \alpha \neq_e \beta \rangle \rrbracket^{\mathfrak{M}} &= \{m \mid \text{there are } n, l \text{ s.t.} \\
&\quad (m, n) \in \llbracket \alpha \rrbracket^{\mathfrak{M}} \text{ and } (m, l) \in \llbracket \beta \rrbracket^{\mathfrak{M}} \text{ and} \\
&\quad \text{data}_e(n) \neq \text{data}_e(l)\}.
\end{aligned}$$

Note that the interpretation $\llbracket \alpha \rrbracket^{\mathfrak{M}}$ of a path expression is a partial function on N , whereas the interpretation $\llbracket \varphi \rrbracket^{\mathfrak{M}}$ of a node expression is a subset of N . A node expression φ is satisfiable iff there is a data model $\mathfrak{M}$ such that $\llbracket \varphi \rrbracket^{\mathfrak{M}} \neq \emptyset$. Finally, we write $\mathfrak{M}, n \models \varphi$ (resp. $\mathfrak{M}, n, m \models \alpha$) iff $n \in \llbracket \varphi \rrbracket^{\mathfrak{M}}$ (resp. $(n, m) \in \llbracket \alpha \rrbracket^{\mathfrak{M}}$).

3 Description Logics with Concrete Domains

In this section we introduce DL with concrete domains. More precisely, we present the logic $\mathcal{ALC}(\mathcal{D})$ [?]. This logic can be seen as the logic $\mathcal{ALC}$ extended with: *abstract features*, *concrete features*, and, more importantly, a *concrete domain* $\mathcal{D}$ (which can be seen as a parameter of the logic). We make these ideas precise below.

Definition 4 (Concrete Domain). A concrete domain is a pair $\mathcal{D} = \langle \Delta_{\mathcal{D}}, \Phi_{\mathcal{D}} \rangle$ where: $\Delta_{\mathcal{D}}$ is a non-empty set, and $\Phi_{\mathcal{D}}$ is a set of n -ary predicate names s.t. each

$P \in \Phi_{\mathcal{D}}$ has an extension $P^{\mathcal{D}} \subseteq \Delta_{\mathcal{D}}^n$. A concrete domain is admissible if: (i) $\Phi_{\mathcal{D}}$ it is closed under negation, i.e., for each $P \in \Phi_{\mathcal{D}}$ there is $\bar{P} \in \Phi_{\mathcal{D}}$ such that $\bar{P}^{\mathcal{D}} = \Delta_{\mathcal{D}} \setminus P^{\mathcal{D}}$, (ii) there exists $\mathbf{1} \in \Phi_{\mathcal{D}}$ s.t. $\mathbf{1}^{\mathcal{D}} = \Delta_{\mathcal{D}}$, and (iii) there exists an algorithm for deciding satisfiability of finite conjunctions of the form $\bigwedge \{P_i(\mathbf{x}_i) \mid i \in [1 \dots k]\}$, where $P_i \in \Phi_{\mathcal{D}}$ and $\mathbf{x}_i$ is a sequence of variables whose length is the arity of P_i .

Below we assume that $\mathcal{D}$ is an arbitrary but fixed concrete domain. Moreover, we assume that $N_C = \{C, D, C_1, C_2, \dots\}$ is a countable set of *concept names*, $N_R = \{r, r_1, r_2, \dots, r, \dots\}$ is a countable set of *role names*, $N_{aF} = \{f, f_1, f_2, \dots\} \subseteq N_R$ is a countable set of *abstract features*, and $N_{cF} = \{g, g_1, g_2, \dots\}$ is a countable set of *concrete features*. The elements in N_{aF}^* are called *abstract paths*.

Definition 5 (Language). The well-formed expressions of $\mathcal{ALC}(\mathcal{D})$, called concepts, are defined by the grammar:

$$C, D ::= C_i \mid \neg C \mid C \sqcap D \mid \exists R.C \mid \exists u_1 g_1 \dots u_n g_n.P,$$

where $C_i \in N_C$, $R \in N_R$, $P \in \Phi_{\mathcal{D}}$ is a n -ary predicate name, each u_i is an abstract path, and $g_i \in N_{cF}$.

Models of this logic are called *interpretations*. We will show later on the correspondence between data models and interpretations.

Definition 6 (Interpretation). An interpretation is a structure defined as $\mathcal{I} = \langle \Delta_{\mathcal{I}}, \cdot^{\mathcal{I}} \rangle$ where:

1. $\Delta_{\mathcal{I}}$ is a non-empty set of elements (also called domain).
2. $\cdot^{\mathcal{I}}$ is a function that maps:
 - (a) every $C_i \in N_C$ to a subset $C_i^{\mathcal{I}} \subseteq \Delta_{\mathcal{I}}$;
 - (b) every $r_i \in N_R$ to a binary relation $r_i^{\mathcal{I}}$ on $\Delta_{\mathcal{I}}$;
 - (c) every $f_i \in N_{aF}$, to a functional binary relation $r_i^{\mathcal{I}}$ on $\Delta_{\mathcal{I}}$;
 - (d) every $g_i \in N_{cF}$, to a partial function $g^{\mathcal{I}} : \Delta_{\mathcal{I}} \rightarrow \Delta_{\mathcal{D}}$.

For every abstract path $u = f_1 \dots f_k$, the interpretation $u^{\mathcal{I}}$ of u is defined as the (relational) composition $f_1^{\mathcal{I}}; \dots; f_k^{\mathcal{I}}$.

Definition 7 (Semantics). Let $\mathcal{I} = \langle \Delta_{\mathcal{I}}, \cdot^{\mathcal{I}} \rangle$ be an interpretation. The semantics of $\mathcal{ALC}(\mathcal{D})$ concepts is defined as:

$$\begin{aligned} (\neg C)^{\mathcal{I}} &= \Delta_{\mathcal{I}} \setminus C^{\mathcal{I}} \\ (C \sqcap D)^{\mathcal{I}} &= C^{\mathcal{I}} \cap D^{\mathcal{I}} \\ (\exists R.C)^{\mathcal{I}} &= \{x \mid \text{exists } y \text{ s.t. } (x, y) \in R^{\mathcal{I}} \text{ and } y \in C^{\mathcal{I}}\} \\ (\exists u_1 g_1 \dots u_n g_n.P)^{\mathcal{I}} &= \{x \mid \text{exists } y_i \in \text{dom}(g_i^{\mathcal{I}}) \text{ s.t. } (x, y_i) \in u_i^{\mathcal{I}} \\ &\quad \text{and } (g_1^{\mathcal{I}}(y_1) \dots g_n^{\mathcal{I}}(y_n)) \in P^{\mathcal{D}}\}. \end{aligned}$$

The results listed below are relevant to provide some complexity bounds of reasoning tasks for DXPath_1^n in a DL setting. See [?] for further details.

Proposition 1 ([?]). *If $\mathcal{D}$ -satisfiability is in PSpace , then we have the following complexity bounds for $\mathcal{ALC}(\mathcal{D})$ reasoning problems:*

1. *concept satisfiability is PSpace -complete;*
2. *ABox consistency is PSpace -complete;*
3. *concept satisfiability w.r.t. acyclic TBoxes is in NExpTime .*

4 From DXPath_1^n to $\mathcal{ALC}(\mathcal{D})$

In this section we show how to characterize DXPath_1^n as a fragment of $\mathcal{ALC}(\mathcal{D})$. The connection between the Boolean components is straightforward. Path expressions of DXPath_1^n can be naturally translated as abstract paths. The interesting element in the translation is the use of concrete features to encode data values and the inclusion of an equality predicate in the concrete domain as a way to encode data comparisons. It turns out that to map DXPath_1^n as a fragment of $\mathcal{ALC}(\mathcal{D})$ we need very simple (admissible) concrete domains of the form $\mathcal{D} = \langle D, \{=, \neq, \mathbf{0}, \mathbf{1}\} \rangle$; where D captures the universe of data values, and $=$ and $\neq$ capture equality and inequality on D . We refer to these simple concrete domains as *equality domains*.

The translation from DXPath_1^n expressions to $\mathcal{ALC}(\mathcal{D})$ concepts is given below.

Definition 8. *The translation function τ from node expressions of DXPath_1^n into concepts of $\mathcal{ALC}(\mathcal{D})$ is defined recursively as:*

$$\begin{aligned} \tau(a_i) &= f_i & \tau(p_i) &= C_i \\ \tau(\alpha\beta) &= \tau(\alpha) \tau(\beta) & \tau(\neg\varphi) &= \neg \tau(\varphi) \\ \tau(\langle \alpha =_{e_i} \beta \rangle) &= \exists \tau(\alpha) g_i, \tau(\beta) g_i. = & \tau(\varphi \wedge \psi) &= \tau(\varphi) \sqcap \tau(\psi) \\ \tau(\langle \alpha \neq_{e_i} \beta \rangle) &= \exists \tau(\alpha) g_i, \tau(\beta) g_i. \neq \end{aligned}$$

To show correctness of the translation, we need to put models of DXPath_1^n and interpretations from $\mathcal{ALC}(\mathcal{D})$ in correspondence.

Definition 9. *Let $\mathfrak{M} = \langle N, D, \{R_a\}_{a \in \text{Mod}}, V, \{\text{data}_e\}_{e \in \text{Cmp}} \rangle$ be a data model. The equality domain $\mathcal{D}^{\mathfrak{M}} = \langle D, \{=, \neq, \mathbf{0}, \mathbf{1}\} \rangle$ is s.t. the extension of $=$ is equality on D , the extension of $\neq$ is inequality on D , the extension of $\mathbf{0}$ is $\emptyset$, and the extension of $\mathbf{1}$ is D . Then, define $\mathfrak{I}_{\mathfrak{M}} = \langle N, \cdot^{\mathfrak{I}_{\mathfrak{M}}} \rangle$ where:*

1. $C_i^{\mathfrak{I}_{\mathfrak{M}}} = \llbracket p_i \rrbracket^{\mathfrak{M}}$
2. $r_i^{\mathfrak{I}_{\mathfrak{M}}} = \emptyset$
3. $f_i^{\mathfrak{I}_{\mathfrak{M}}} = \llbracket a_i \rrbracket^{\mathfrak{M}}$
4. $g_i^{\mathfrak{I}_{\mathfrak{M}}}(n) = \text{data}_{e_i}(n)$.

The following result states that data models of DXPath_1^n can be seen as interpretations of $\mathcal{ALC}(\mathcal{D})$ where $\mathcal{D}$ is an equality domain.

Lemma 1. $\mathfrak{I}_{\mathfrak{M}}$ is a interpretation of $\mathcal{ALC}(\mathcal{D})$ and $\mathcal{D}^{\mathfrak{M}}$ is an admissible concrete domain.

The proof of the lemma above is straightforward, and enables us to establish the following correspondence between the two logics.

Theorem 1. Let φ be a node expression, α a path expression, and $\mathfrak{M}$ a data model, then $\llbracket \alpha \rrbracket^{\mathfrak{M}} = \tau(\alpha)^{\mathfrak{I}_{\mathfrak{M}}}$ and $\llbracket \varphi \rrbracket^{\mathfrak{M}} = \tau(\varphi)^{\mathfrak{I}_{\mathfrak{M}}}$.

Proof. Let $\mathfrak{M} = \langle N, D, \{R_a\}_{a \in \text{Mod}}, V, \{\text{data}_e\}_{e \in \text{Cmp}} \rangle$, and $n \in N$. The proof is by induction on φ and α . Base cases and Boolean inductive cases are direct by definition. We show below the case for $=_e$ data comparisons (the case of $\neq_e$ being analogous).

($\varphi = \langle \alpha =_{e_i} \beta \rangle$). We prove only the left-to-right direction. The right-to-left direction is analogous. Let $n \in \llbracket \langle \alpha =_{e_i} \beta \rangle \rrbracket^{\mathfrak{M}}$. By ??, there are x and y s.t. $(n, x) \in \llbracket \alpha \rrbracket^{\mathfrak{M}}$, $(n, y) \in \llbracket \beta \rrbracket^{\mathfrak{M}}$, and $\text{data}_{e_i}(x) = \text{data}_{e_i}(y)$. By ??, we know (1) $(x, \text{data}_{e_i}(x)) \in (g_i)^{\mathfrak{I}_{\mathfrak{M}}}$ and (2) $(y, \text{data}_{e_i}(y)) \in (g_i)^{\mathfrak{I}_{\mathfrak{M}}}$. From the IH, (3) $(n, x) \in \tau(\alpha)^{\mathfrak{I}_{\mathfrak{M}}}$ and (4) $(n, y) \in \tau(\beta)^{\mathfrak{I}_{\mathfrak{M}}}$. Using (1) and (3), we obtain $(n, \text{data}_{e_i}(x)) \in (\tau(\alpha)g_i)^{\mathfrak{I}_{\mathfrak{M}}}$. Using (2) and (4), we obtain $(n, \text{data}_{e_i}(y)) \in (\tau(\beta)g_i)^{\mathfrak{I}_{\mathfrak{M}}}$. Therefore, $n \in (\exists \tau(\alpha)g_i, \tau(\beta)g_i. =)^{\mathfrak{I}_{\mathfrak{M}}}$. Finally, by ??, $n \in \tau(\langle \alpha =_{e_i} \beta \rangle)^{\mathfrak{I}_{\mathfrak{M}}}$. $\square$

The result in ?? tells us that DXPath_1^n forms a very simple fragment of $\mathcal{ALC}(\mathcal{D})$. This result further makes it possible to define and study the properties of standard reasoning methods for DXPath_1^n in a DL setting. In particular, we obtain complexity upper bounds for them, by ?. However these bounds are not optimal. The next section is devoted to show tight complexity bounds for DXPath_1^n .

5 Complexity of Reasoning in DXPath_1^n

In this section we explore the computational complexity of some typical DL-reasoning tasks for the case of DXPath_1^n . We pay attention to the complexity of *concept satisfiability*, *ABox consistency* and *concept satisfiability w.r.t. acyclic TBoxes* for DXPath_1^n expressions. In particular, we will show that DXPath_1^n can be actually seen as a fragment of *alcd* enjoying nice computational properties.

5.1 DXPath_1^n satisfiability

We start by proving that the satisfiability problem for node expressions in DXPath_1^n (referred to as *concept satisfiability* in the DL terminology) is NP-complete. The argument of the proof is akin to that for the modal logic $K_n\text{Alt}_1$. The key is a *selection function* (see [?, Th. 6.37]). In what follows, we will use $*$ to refer indistinctly to $=$ or $\neq$ in data comparisons.

Definition 10. Let $\mathfrak{M} = \langle N, D, \{R_a\}_{a \in \text{Mod}}, V, \{\text{data}_e\}_{e \in \text{Cmp}} \rangle$ be a data model, and $n \in N$. The selection function s is recursively defined over path expressions and node expressions as:

$$\begin{aligned} s(a, n) &= \begin{cases} \{n\} & \text{if } R_a(n) \text{ is undefined} \\ \{n, R_a(n)\} & \text{if } R_a(n) \text{ is defined} \end{cases} \\ s(a\alpha, n) &= \begin{cases} \{n\} & \text{if } R_a(n) \text{ is undefined} \\ \{n\} \cup s(\alpha, R_a(n)) & \text{if } R_a(n) \text{ is defined} \end{cases} \\ s(p, n) &= \{n\} \\ s(\neg\varphi, n) &= s(\varphi, n) \\ s(\varphi \wedge \psi, n) &= s(\varphi, n) \cup s(\psi, n) \\ s(\langle \alpha *_{\mathbf{e}} \beta \rangle, n) &= s(\alpha, n) \cup s(\beta, n) \end{aligned}$$

Then, define $\mathfrak{M}|_{s(\varphi, n)} = \langle N', D, \{R'_a\}_{a \in \text{Mod}}, V', \{\text{data}'_e\}_{e \in \text{Cmp}} \rangle$ as:

1. $N' = N \cap s(\varphi, n)$;
2. for all $a \in \text{Mod}$, $R'_a = R_a \cap (N' \times N')$
3. $V : \text{Prop} \rightarrow 2^{N'}$ is s.t.: $V'(p) = V(p) \cap N'$
4. for all $e \in \text{Cmp}$, $\text{data}'_e : N' \rightarrow D$ is s.t.: $\text{data}'_e(n) = \text{data}_e(n)$.

We refer to $\mathfrak{M}|_{s(\varphi, n)}$ as the selected data model.

In a nutshell, given a model, s selects the relevant pieces on which to evaluate a given node expression. The next lemma states that this selection preserves the truth value of a node expression.

Lemma 2. Let φ be a node expression. Moreover, let $\mathfrak{M} = \langle N, D, \{R_a\}_{a \in \text{Mod}}, V, \{\text{data}_e\}_{e \in \text{Cmp}} \rangle$ be s.t. $\llbracket \varphi \rrbracket^{\mathfrak{M}} \neq \emptyset$. Then, for all $\{n, m\} \subseteq N$, all path expressions α , and all node expressions φ , the following statements are true:

1. $(n, m) \in \llbracket \alpha \rrbracket^{\mathfrak{M}}$ iff $(n, m) \in \llbracket \alpha \rrbracket^{\mathfrak{M}|_{s(\alpha, n)}}$;
2. $n \in \llbracket \varphi \rrbracket^{\mathfrak{M}}$ iff $n \in \llbracket \varphi \rrbracket^{\mathfrak{M}|_{s(\varphi, n)}}$.

Moreover, $|s(\varphi, n)| \leq |\varphi| + 1$.

Proof. The proof is by induction on α and φ .

Base cases.

- $(\alpha = a)$. Left-to-right: let $(n, m) \in \llbracket a \rrbracket^{\mathfrak{M}}$, i.e., $(n, m) \in R_a$. We know $s(a, n) = \{n, R_a(n)\} = \{n, m\}$. Then, $(n, m) \in (R_a \cap ((s(a, n))^2))$. So, $(n, m) \in \llbracket a \rrbracket^{\mathfrak{M}|_{s(a, n)}}$. The right-to-left direction is similar.
- $(\varphi = p)$. Left-to-right: let $n \in \llbracket p \rrbracket^{\mathfrak{M}}$, i.e., $n \in V(p)$. Then, $n \in (V(p) \cap s(p, n))$. So, $n \in \llbracket p \rrbracket^{\mathfrak{M}|_{s(p, n)}}$. The right-to-left direction is similar.

Inductive cases.

- $(\alpha = a\alpha)$. Let $(n, m) \in \llbracket a\alpha \rrbracket^{\mathfrak{M}}$. We know $(n, R_a(n)) \in R_a$ and $(R_a(n), m) \in \llbracket \alpha \rrbracket^{\mathfrak{M}}$. The former implies $(n, R_a(n)) \in (R_a \cap s(a, n)^2)$. By the IH, the latter implies $(R_a(n), m) \in \llbracket \alpha \rrbracket^{\mathfrak{M}|_{s(\alpha, n)}}$. From the def. of s , we know $s(a\alpha, n) = \{n\} \cup s(\alpha, R_a(n))$. Then, $(n, m) \in (R_a \cap s(a\alpha, n)^2)$. Therefore, $(n, m) \in \llbracket a\alpha \rrbracket^{\mathfrak{M}|_{s(a\alpha, n)}}$. The other direction is similar.

$(\varphi = \neg\psi \text{ and } \varphi = \psi \wedge \chi)$. Immediate from the IH.
 $(\varphi = \langle \alpha =_e \beta \rangle)$. Let $n \in \llbracket \langle \alpha =_e \beta \rangle \rrbracket^{\mathfrak{M}}$. We know there are m, m' s.t.: $(n, m) \in \llbracket \alpha \rrbracket^{\mathfrak{M}}$, $(n, m') \in \llbracket \beta \rrbracket^{\mathfrak{M}}$, and $\text{data}_e(m) = \text{data}_e(m')$. By IH, $(n, m) \in \llbracket \alpha \rrbracket^{\mathfrak{M} \upharpoonright_{s(\alpha, n)}}$, and $(n, m') \in \llbracket \beta \rrbracket^{\mathfrak{M} \upharpoonright_{s(\beta, n)}}$. From the def. of s , it follows that $\{n, m, m'\} \subseteq s(\langle \alpha =_e \beta \rangle, n)$. Then, $n \in \llbracket \langle \alpha =_e \beta \rangle \rrbracket^{\mathfrak{M} \upharpoonright_{s(\langle \alpha =_e \beta \rangle, n)}}$. The other direction is similar.
 $(\varphi = \langle \alpha \neq_e \beta \rangle)$. Identical to the one above.

It remains to show that $|s(\varphi, n)| \leq |\varphi| + 1$, where $|\varphi|$ is the number of modal symbols of φ . The proof is by induction on φ . Notice that s adds new nodes only while “breaking down” path expressions. More precisely, s adds only one node per each modal symbol in the recursive case. The interesting case is $\varphi = \langle \alpha *_e \beta \rangle$. In this case, we have $|s(\varphi, n)| \leq |s(\alpha, n)| + |s(\beta, n)|$. Immediately, $|s(\alpha, n)| \leq |\alpha| + 1$, and $|s(\beta, n)| \leq |\beta| + 1$. In both cases the $+1$ corresponds to the node n they have in common. Thus, $|s(\varphi, n)| \leq |\varphi| + 1$ as desired. $\square$

We are now in position to characterize the complexity of the satisfiability problem for DXPath_1^n .

Theorem 2. *The satisfiability problem for node expressions in DXPath_1^n is NP-complete.*

Proof. Given a node expression φ , if it is satisfiable by ?? we can guess a data model $\mathfrak{M}$ that is polynomial on $|\varphi|$, and check whether $\llbracket \varphi \rrbracket^{\mathfrak{M}} \neq \emptyset$. The last check can be done in polynomial time (see the result in [?]). Moreover, checking that $\mathfrak{M}$ is a proper data model can also be done in polynomial time. Thus, the satisfiability problem for DXPath_1^n is in NP. Hardness is immediate from the satisfiability problem of propositional logic. $\square$

5.2 Reasoning with DXPath_1^n ABoxes and TBoxes

We now turn to ABox consistency, and concept satisfiability w.r.t. acyclic TBoxes. In what follows, we adopt the DL terminology and refer to proposition symbols as concept names and to node expressions as concepts. We also consider a countable set of *nominals* Nom , which corresponds in the DL terminology to *individual names*. We begin with some preliminary definitions.

Definition 11. *A terminological box (TBox) in DXPath_1^n is a set $\mathcal{T}$ of expressions of the form $p \doteq \psi$, where p is a concept name, and ψ is a concept. We call $p \doteq \psi$ a concept definition. We say that a concept name p directly uses a concept φ if there is a concept definition $p \doteq \psi$ such that φ appears in ψ . We say that p uses φ iff φ is in the transitive closure of the ‘directly uses’ relation. Moreover, $\mathcal{T}$ is acyclic iff*

1. *no concept name uses itself, and*
2. *the left-side of the concept definitions are all pair-wise disjoint.*

If $\mathcal{T}$ is acyclic and $p \doteq \varphi \in \mathcal{T}$, we say that φ defines p and call φ the definition of p .

TBoxes describe general relations between the concepts, in this case, providing definitions that can be globally assumed. On the other hand, ABoxes describe properties about particular individuals.

Definition 12. An assertion box (ABox) in DXPath_1^n is a set $\mathcal{A}$ of expressions of the form:

$$i : \varphi, \quad (i, j) : \alpha, \quad i =_e j, \quad \text{or} \quad i \neq_e j,$$

where $i, j \in \text{Nom}$, and $\alpha \in \text{Mod}^+$.

To investigate reasoning problems involving ABoxes and TBoxes, we introduce how to interpret them on a model. Note that for ABoxes we need to extend models with an interpretation for nominals.

Definition 13. The satisfaction relation for TBoxes is given w.r.t. a data model $\mathfrak{M} = \langle N, D, \{R_a\}_{a \in \text{Mod}}, V, \{\text{data}_e\}_{e \in \text{Cmp}} \rangle$, and for ABoxes w.r.t. $\mathfrak{M}$ and a function $\text{nom} : \text{Nom} \rightarrow N$. More precisely:

$$\begin{aligned} \mathfrak{M} \models p \doteq \psi & \quad \text{iff} \quad \llbracket p \rrbracket^{\mathfrak{M}} = \llbracket \psi \rrbracket^{\mathfrak{M}} \\ \mathfrak{M}, \text{nom} \models i : \varphi & \quad \text{iff} \quad \text{nom}(i) \in \llbracket \varphi \rrbracket^{\mathfrak{M}} \\ \mathfrak{M}, \text{nom} \models (i, j) : \alpha & \quad \text{iff} \quad (\text{nom}(i), \text{nom}(j)) \in \llbracket \alpha \rrbracket^{\mathfrak{M}} \\ \mathfrak{M}, \text{nom} \models i =_e j & \quad \text{iff} \quad \text{data}_e(\text{nom}(i)) = \text{data}_e(\text{nom}(j)) \\ \mathfrak{M}, \text{nom} \models i \neq_e j & \quad \text{iff} \quad \text{data}_e(\text{nom}(i)) \neq \text{data}_e(\text{nom}(j)). \end{aligned}$$

Let $\mathcal{T}$ be a TBox. We write $\mathfrak{M} \models \mathcal{T}$ iff for all $p \doteq \varphi \in \mathcal{T}$, it follows that $\mathfrak{M} \models p \doteq \varphi$. We write $\mathfrak{M}, \text{nom} \models \mathcal{A}$ iff for all $\xi \in \mathcal{A}$, $\mathfrak{M}, \text{nom} \models \xi$. We say that an ABox $\mathcal{A}$ is consistent iff exists $\mathfrak{M}, \text{nom}$ such that $\mathfrak{M}, \text{nom} \models \mathcal{A}$.

Reasoning with TBoxes and ABoxes is *global*, i.e., expresions are either true or false everywhere in the model. This differs from the *local* reasoning from ??, in which the truth or falsity of an expression depends on the particular node being considered. This jump from local reasoning to global reasoning, common in DL, in many cases, causes a jump in the complexity of the reasoning tasks. In the rest of this section we analyze the effects of adding TBoxes and ABoxes in DXPath_1^n from a computational complexity perspective.

The first result in this section brings back into the picture the selection function s in ??. It is not hard to see how s can be extended to handle nominals. We would simply need to add cases $s(i : \varphi, n) = s(\varphi, \text{nom}(i))$ and $s((i, j) : \alpha, n) = s(\alpha, \text{nom}(i))$. With this new definition of the selection function, we obtain the result below by adapting the argument used in the proof of ??.

Theorem 3. The ABox consistency problem for DXPath_1^n is NP-complete.

Let us now turn our attention to concept satisfiability w.r.t. a TBox. We begin by making clear the problem we will tackle.

Definition 14. We say that a concept name p is satisfiable w.r.t. a TBox $\mathcal{T}$ iff there is $\mathfrak{M}$ such that $\mathfrak{M} \models \mathcal{T}$ and $\llbracket p \rrbracket^{\mathfrak{M}} \neq \emptyset$.

It is easily seen that the problem of concept name satisfiability can be extended to the problem of *arbitrary concept satisfiability*. More precisely, an arbitrary concept φ is *satisfiable* w.r.t. a TBox $\mathcal{T}$ iff there is $\mathfrak{M}$ s.t. $\mathfrak{M} \models \mathcal{T}$ and $\llbracket \varphi \rrbracket^{\mathfrak{M}} \neq \emptyset$. This problem of arbitrary concept satisfiability can be reformulated as a problem of concept name satisfiability. It can be proven that φ is satisfiable w.r.t. $\mathcal{T}$ iff p is satisfiable w.r.t. $\mathcal{T} \cup \{p \doteq \varphi\}$, for p a new concept name, i.e., p not in $\mathcal{T}$ and φ . The following observation is also important. Suppose that we wish to check whether a concept name p is satisfiable w.r.t. a TBox $\mathcal{T}$. W.l.o.g., we can always assume there is $p \doteq \varphi \in \mathcal{T}$. It can be proven that if there is no $p \doteq \varphi \in \mathcal{T}$, then, p is satisfiable w.r.t. $\mathcal{T}$ iff p is satisfiable w.r.t. $\mathcal{T} \cup \{p \doteq q \vee \neg q\}$, for q a new concept name.

The problem of concept name satisfiability w.r.t. acyclic TBoxes assumes the TBox is *acyclic*. This restriction to acyclic TBoxes allows us to reformulate the definition of the selection function in ??, and of *selected model*, to obtain an NP upper bound. The crucial point in this reformulation is the definition of the valuation function for the selected model, taking into consideration the TBox. We make these ideas precise below.

Definition 15. We call a TBox $\mathcal{T}$ *simple* iff it is *acyclic* and:

1. for all $p \doteq \psi \in \mathcal{T}$, $\psi \in \{\neg q, q \wedge r, q \vee r, \langle \alpha *_e \beta \rangle, \neg \langle \alpha *_e \beta \rangle\}$;
2. for all $\{p \doteq \neg q, p' \doteq \psi\} \subseteq \mathcal{T}$, we have $p' \neq q$,

where $p, q, r, p' \in \text{Prop}$.

Simple TBoxes form an interesting class of acyclic TBoxes, since, as stated in ??, they preserve “size” and “meaning” of acyclic TBoxes.

Lemma 3. Any *acyclic* TBox $\mathcal{T}$ can be converted in polynomial time into a *simple* TBox $\mathcal{T}'$ s.t. for all $\mathfrak{M} \models \mathcal{T}$, there is $\mathfrak{M}' \models \mathcal{T}'$ agreeing with $\mathfrak{M}$ on all concept names in $\mathcal{T}$, and viceversa.

Proof. The proof follows similar arguments as that for $\mathcal{ALC}$ found in [?], but it needs to be extended for the case of data comparisons. Let $\text{nnf}(\psi)$ be the *negation normal form* (NNF) of ψ , i.e., negation only occurs at the atomic level. Every node expression in DXPath_1^n can be transformed into a node expression in NNF in linear time. Apply the following rules until no more rules can be applied:

Rule 1: Eliminate non-atomic negations. Convert all $p \doteq \psi \in \mathcal{T}$ into $p \doteq \text{nnf}(\psi)$. Then, add $p' \doteq \text{nnf}(\neg \psi)$ for $p' \in \text{Prop}$ new, and replace each occurrence of $\neg p$ with p' in $\mathcal{T}$. Since α and β do not contain negations, $\neg \langle \alpha *_e \beta \rangle$ is already in NNF.

Rule 2: Break definitions. For every φ that is not a concept name, convert all $p \doteq \varphi \wedge \psi \in \mathcal{T}$ into $p \doteq p' \wedge \psi$, and add $p' \doteq \varphi$, where $p' \in \text{Prop}$ is new each time. Do the same for $p \doteq \psi \wedge \varphi$ and for disjunctions.

Rule 3: Eliminate redundant definitions. Eliminate all $p \doteq p'$, and replace each occurrence of p' with p in $\mathcal{T}$.

The rules 1 to 3 above terminate in polynomial time and create a new TBox $\mathcal{T}'$ which is polynomial on the size of $\mathcal{T}$. It is immediate to see that the obtained TBox $\mathcal{T}'$ is simple. The proof that for every data model $\mathfrak{M}$ of $\mathcal{T}$ there is a data model $\mathfrak{M}'$ of $\mathcal{T}'$ that agrees with $\mathfrak{M}$ on all the proposition symbols used in $\mathcal{T}$, and viceversa, builds on the argument found in [?,?]. $\square$

The result in ?? tells us that in the context of acyclic TBoxes there is no loss of generality in working with simple TBoxes. We are now in a position to define the selection function needed to establish the main result of this section.

Definition 16. Let $\mathcal{T}$ be a simple TBox. For every data model $\mathfrak{M} = \langle N, D, \{R_a\}_{a \in \text{Mod}}, V, \{\text{data}_e\}_{e \in \text{Cmp}} \rangle$ and $n \in N$, define the selection function s exactly as in ?? except that:

$$\begin{aligned} s(a, n) &= \begin{cases} \{n\} & \text{if } R_a(n) \text{ is undefined} \\ \{n, R_a(n)\} & \text{if } R_a(n) \text{ is defined} \end{cases} \\ s(a\alpha, n) &= \begin{cases} \{n\} & \text{if } R_a(n) \text{ is undefined} \\ \{n\} \cup s(\alpha, R_a(n)) & \text{if } R_a(n) \text{ is defined} \end{cases} \\ s(p, n) &= \begin{cases} \{n\} & \text{if there is no } p \doteq \psi \in \mathcal{T} \\ s(\psi, n) & \text{if there is } p \doteq \psi \in \mathcal{T} \end{cases} \\ s(\neg\varphi, n) &= s(\varphi, n) \\ s(\varphi \wedge \psi, n) &= s(\varphi, n) \cup s(\psi, n) \\ s(\varphi \vee \psi, n) &= s(\varphi, n) \cup s(\psi, n) \\ s(\langle \alpha *_e \beta \rangle, n) &= s(\alpha, n) \cup s(\beta, n). \end{aligned}$$

Then, define $\mathfrak{M}|_{s(\varphi, n)} = \langle N', D, \{R'_a\}_{a \in \text{Mod}}, V', \{\text{data}'_e\}_{e \in \text{Cmp}} \rangle$ as:

1. $N' = N \cap s(\varphi, n)$;
2. for all $a \in \text{Mod}$, $R'_a = R_a \cap (N' \times N')$
3. $V : \text{Prop} \rightarrow 2^{N'}$ is s.t.:

$$V'(p) = \begin{cases} \{n\} & \text{if there is no } p \doteq \psi \in \mathcal{T} \\ \emptyset & \text{if there is } q \doteq \neg p \in \mathcal{T} \\ \llbracket \langle \alpha *_e \beta \rangle \rrbracket^{\mathfrak{M}} \cap N' & \text{if there is } p \doteq \langle \alpha *_e \beta \rangle \in \mathcal{T} \\ \llbracket \neg[\alpha *_e \beta] \rrbracket^{\mathfrak{M}} \cap N' & \text{if there is } p \doteq [\alpha *_e \beta] \in \mathcal{T} \\ N' \setminus V'(q) & \text{if there is } p \doteq \neg q \in \mathcal{T} \\ V'(q) \cap V'(q') & \text{if there is } p \doteq q \wedge q' \in \mathcal{T} \\ V'(q) \cup V'(q') & \text{if there is } p \doteq q \vee q' \in \mathcal{T} \end{cases}$$

4. for all $e \in \text{Cmp}$, $\text{data}'_e : N' \rightarrow D$ is s.t.: $\text{data}'_e(n) = \text{data}_e(n)$.

We call $\mathfrak{M}|_{s(\varphi, n)}$ the selected data model.

The first distinction between the definition above and the one in ?? is the case of propositional symbols. Here, if a propositional symbol p is such that $p \doteq \psi \in \mathcal{T}$, we need to proceed in unfolding such a definition recursively. Even

though a TBox provides global information about the model, we only unfold the definitions that are relevant from the evaluation point, thus, we only take into account definitions in the TBox locally. The second distinction is a minor one, and concerns the definition of a case of disjunction. We need this case now, as in simple TBoxes we deal with formulas in negation normal form.

It is important to notice how, contrary to the simplicity in the definition of the selected model in ??, the definition of the valuation function for the selected model for the case of TBoxes requires some ingenuity and takes advantage of the structure of the TBox. In particular, the structure of the TBox guarantees that the recursive definition of V' is well-founded. More interesting is the definition of the base cases for V' . These distinguish the cases in which a concept name p is defined in terms of a diamond or box comparison, or it is undefined. In the latter case, it further distinguishes whether p appears negated or not. The definition of the valuation for p in each of these cases plays a crucial role in the proof of the following result.

Lemma 4. *Let $\mathcal{T} \cup \{p \doteq \psi\}$ be a simple TBox. Moreover, let $\mathfrak{M}$ be s.t. $\mathfrak{M} \models \mathcal{T}$ and $n \in \llbracket p \rrbracket^{\mathfrak{M}}$. The following are true statements:*

1. $\mathfrak{M}|_{s(p,n)} \models \mathcal{T}$,
2. $n \in \llbracket p \rrbracket^{\mathfrak{M}|_{s(p,n)}}$, and
3. $|s(p,n)|$ is polynomial in $|\mathcal{T}|$.

Proof. Item 1 is immediate. From the way in which V' is defined, we know that for each concept name q s.t. $q \doteq \psi \in \mathcal{T}$ it happens that $\llbracket q \rrbracket^{\mathfrak{M}|_{s(p,n)}} = \llbracket \psi \rrbracket^{\mathfrak{M}|_{s(p,n)}}$. This establishes $\mathfrak{M}|_{s(p,n)} \models \mathcal{T}$.

The following terminology is useful for proving Items 2 and 3. We call ψ a *basic* definition of q iff $\psi \in \{\neg r, \langle \alpha *_{\mathbf{e}} \beta \rangle, [\alpha *_{\mathbf{e}} \beta]\}$; otherwise we call ψ a *compound* definition of q . The proof of Items 2 and 3 are by induction on the structure of the definition ψ of p .

For Item 2:

Bases cases.

- $(\psi \in \{\langle \alpha *_{\mathbf{e}} \beta \rangle, [\alpha *_{\mathbf{e}} \beta]\})$. We know that $\llbracket p \rrbracket^{\mathfrak{M}} = \llbracket \psi \rrbracket^{\mathfrak{M}}$. Then, $n \in \llbracket \psi \rrbracket^{\mathfrak{M}}$, and, from the def. of s , $n \in s(\psi, n)$. This implies $n \in \llbracket \psi \rrbracket^{\mathfrak{M}|_{s(p,n)}}$. Finally, $n \in \llbracket p \rrbracket^{\mathfrak{M}|_{s(p,n)}}$ from the def. of V' .
- $(\psi = \neg r)$. We know that $p \doteq \neg r \in \mathcal{T}$. From the def. of V' , $\llbracket r \rrbracket^{\mathfrak{M}|_{s(p,n)}} = \emptyset$, and $\llbracket p \rrbracket^{\mathfrak{M}|_{s(p,n)}} = \llbracket \neg r \rrbracket^{\mathfrak{M}|_{s(p,n)}}$; i.e., $\llbracket p \rrbracket^{\mathfrak{M}|_{s(p,n)}} = s(p, n)$. Immediately, $n \in \llbracket p \rrbracket^{\mathfrak{M}|_{s(p,n)}}$.

Inductive cases.

$(\psi = r \wedge r')$. The proof is by cases.

Case 1: $(r \doteq \psi \notin \mathcal{T} \text{ and } r' \doteq \psi' \notin \mathcal{T})$. From the def. of V' , $\llbracket r \rrbracket^{\mathfrak{M}|_{s(p,n)}} = \llbracket r \rrbracket^{\mathfrak{M}|_{s(p,n)}} = \{n\}$. We get $n \in \llbracket p \rrbracket^{\mathfrak{M}|_{s(p,n)}}$ immediately from the def. of V' .

Case 2: $(r \doteq \psi \in \mathcal{T} \text{ and } r' \doteq \psi' \notin \mathcal{T})$. From the IH, we know $n \in \llbracket r \rrbracket^{\mathfrak{M}|_{s(p,n)}}$. Moreover, from the def. of V' , $\llbracket r' \rrbracket^{\mathfrak{M}|_{s(p,n)}} = \{n\}$. We get $n \in \llbracket p \rrbracket^{\mathfrak{M}|_{s(p,n)}}$ immediately from the def. of V' .

Case 3: $(r \doteq \psi \in \mathcal{T} \text{ and } r' \doteq \psi' \in \mathcal{T})$. From the IH, $n \in \llbracket r \rrbracket^{\mathfrak{M}|_{s(p,n)}}$ and $n \in \llbracket r' \rrbracket^{\mathfrak{M}|_{s(p,n)}}$. We get $n \in \llbracket p \rrbracket^{\mathfrak{M}|_{s(p,n)}}$ immediately from the def. of V' .

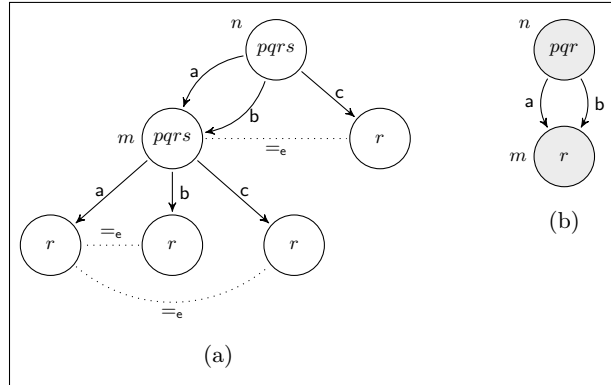


Fig. 2: Model and Selected Model

$(\psi = r \vee r')$. Similar to the case for $\wedge$.

For Item 3:

Bases cases.

$(\psi \in \{\neg r, \langle \alpha *_e \beta \rangle, [\alpha *_e \beta]\})$. If $\psi = \neg r$, then, the result is immediate. The interesting case is $\psi \doteq \langle \alpha *_e \beta \rangle$ (the case $\psi = [\alpha *_e \beta]$ is similar). In this case, notice that $|s(p, n)| = |s(\psi, n)| \leq |\alpha| + |\beta| \leq |\mathcal{T}|$. This observation concludes the proof of the base cases.

Inductive cases.

$(\psi = r \wedge r')$. The proof is by cases (cf. Item 2 above). The interesting case is $r \doteq \psi \in \mathcal{T}$ and $r' \doteq \psi' \in \mathcal{T}$. From the IH, $|s(r, n)|$ and $|s(r', n)|$ are polynomial on $|\mathcal{T}|$. It is immediate that $|s(r, n)| + |s(r', n)|$ is also polynomial on $|\mathcal{T}|$. From this fact, $|s(p, n)|$ is polynomial on $|\mathcal{T}|$ as desired.

$(\psi = r \vee r')$. Similar to the case for $\wedge$. $\square$

?? shows how the selection function works.

Example 1. Consider the TBox

$$\mathcal{T} = \{p \doteq q \wedge r, q \doteq \langle a =_e b \rangle, r \doteq [bc \neq_e c], s \doteq \langle a =_e c \rangle\},$$

and let $\mathfrak{M}$ be the model in ??(a). In this figure, dotted arrows represent the relevant equality information (i.e., data_e applied to those nodes returns equal values, and returns distinct values if applied to any other two different nodes). Immediately, $\mathfrak{M} \models \mathcal{T}$. For instance, $\llbracket q \rrbracket^{\mathfrak{M}} = \{n, m\} = \llbracket \langle a =_e b \rangle \rrbracket^{\mathfrak{M}}$. Similarly, if we take N to be the set of all nodes in $\mathfrak{M}$, $\llbracket r \rrbracket^{\mathfrak{M}} = N = \llbracket [bc \neq_e c] \rrbracket^{\mathfrak{M}}$. Notice also that this model establishes that p is satisfiable w.r.t. $\mathcal{T}$. This is easily verified since $n \in \llbracket p \rrbracket^{\mathfrak{M}} = \llbracket q \wedge r \rrbracket^{\mathfrak{M}} = \llbracket q \rrbracket^{\mathfrak{M}} \cap \llbracket r \rrbracket^{\mathfrak{M}} = \{n, m\}$.

In turn, the model in ??(b) corresponds to $\mathfrak{M}|_{s(p,n)}$. Intuitively, $\mathfrak{M}|_{s(p,n)}$ can be understood as the “relevant part” of $\mathfrak{M}$ which is necessary to check the satisfiability of p . It is worth pointing out that such a “relevant part” is not simply a restriction, it carries with it a change in valuations. For instance, $\llbracket s \rrbracket^{\mathfrak{M}} = \llbracket \langle a =_e c \rangle \rrbracket^{\mathfrak{M}} = \{n, m\}$, whereas $\llbracket s \rrbracket^{\mathfrak{M}|_{s(p,n)}} = \llbracket \langle a =_e c \rangle \rrbracket^{\mathfrak{M}|_{s(p,n)}} = \emptyset$. Notice however, that this change in valuations does not affect the satisfiability of p w.r.t. $\mathcal{T}$. More precisely, it can be checked that $n \in \llbracket p \rrbracket^{\mathfrak{M}|_{s(p,n)}}$ since $\llbracket p \rrbracket^{\mathfrak{M}|_{s(p,n)}} = \llbracket q \wedge r \rrbracket^{\mathfrak{M}|_{s(p,n)}} = \llbracket q \rrbracket^{\mathfrak{M}|_{s(p,n)}} \cap \llbracket r \rrbracket^{\mathfrak{M}|_{s(p,n)}} = \{n\}$.

By using ?? and the same argument as in ??, this time w.r.t. a TBox, we obtain the intended result.

Theorem 4. *The problem of concept satisfiability w.r.t. a TBox in DXPath_1^n is NP-complete.*

6 Discussion and Final Remarks

We investigated the fragment DXPath_1^n of the query language XPath from a DL perspective. This fragment features data comparisons and is interpreted over data graphs with functional accessibility relations. We showed with an example that DXPath_1^n is expressive enough to model interesting scenarios. With this as a starting point, we presented a novel connection between DXPath_1^n and a fragment of $\mathcal{ALC}(\mathcal{D})$. This enabled us to see DXPath_1^n as a DL with a very simple concrete domain and to obtain upper bounds on the complexity for some reasoning tasks via transfer results.

In turn, we showed that these bounds are not tight by establishing that for DXPath_1^n concept satisfiability, ABox consistency and concept satisfiability w.r.t. a TBox are all NP-complete problems. As a contrast, in [?, ?, ?] the following complexity result is stated for *arithmetic* domains $\mathcal{D} = \langle \Delta_{\mathcal{D}}, \Phi_{\mathcal{D}} \rangle$ –i.e., concrete domains where $\Delta_{\mathcal{D}}$ contains the natural numbers and $\Phi_{\mathcal{D}}$ contains unary predicates for equality with zero and with one, a binary equality predicate, and ternary predicates expressing addition and multiplication– using an NExpTime-complete variant of the Post Correspondence Problem.

Proposition 2 ([?, ?, ?]). *For any arithmetic concrete domain $\mathcal{D}$, $\mathcal{ALC}(\mathcal{D})$ -concept satisfiability w.r.t. acyclic TBoxes is NExpTime-hard.*

This is in surprising contrast with the complexity results we established, showing the relevance of the restrictions imposed on the expressive power of DXPath_1^n . In particular, forcing accessibility relations to be partial functions is an important component of our selection function that establishes a small model property for the reasoning tasks we investigated. Liberating this restriction will move us close to the logic $\mathcal{ALCP}(\mathcal{D})$ [?], that allows for the use of roles where $\mathcal{ALC}(\mathcal{D})$ permits only abstract features. It is clear that we could then modify the translation function in ?? to map modal labels a_i to role names r_i in this case. Hence XPath with arbitrary accessibility relations results as a fragment of $\mathcal{ALCP}(\mathcal{D})$. But allowing this additional expressive power might have an important impact on complexity, as the following result suggests:

Proposition 3 ([?]). *For any arithmetic concrete domain $\mathcal{D}$, satisfiability of $\mathcal{ALCP}(\mathcal{D})$ -concepts is NExpTime-hard.*

Notice that the result above concerns *concept satisfiability*, which is usually simpler than dealing with TBoxes. Still, perhaps the additional constraints on node and path expressions (absence of tests, simpler comparisons in the concrete domain) might lead to interesting results in the fragment of $\mathcal{ALCP}(\mathcal{D})$ that concerns XPath over arbitrary accessibility relations.

As we discussed in the introduction, only the purely navigational fragment of XPath has been studied in a DL setting before [?,?]. In this article, we show how to deal with data comparisons by linking XPath with DL with concrete domains. The work we presented here is a first step in this direction. In particular, it would be interesting to see whether our methods and results can be adapted and/or extended to deal with other logics, such as those capturing fragments of the Shapes Constraint Language (SHACL) (see e.g. [?,?] for details). We should also investigate the effect of allowing additional constructs on path expressions like tests, and non-deterministic choice. Finally, we would also like to implement DXPath_1^n reasoners for the tasks we investigated.